Python Project Packaging

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-04-21

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: Start with Testing & Project Feedback	28/4	Y	RZ D8
11: Testing and debugging	5/5	N	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment and peer review **today**.

Plan for today

Theory

1. Structure of a python package
2. Using python packages
3. Creating our own python package
4. Handling Paths in your project

Plan for today

Theory

1. Structure of a python package
2. Using python packages
3. Creating our own python package
4. Handling Paths in your project

If time

1. Team Reflection
2. Discuss the next steps in your project

Python Package Structure

What is a Python package?

A **python package**

What is a Python package?

A **python package**

- enforces logical structuring of your code, breaking it into smaller, manageable modules (reusable).

What is a Python package?

A **python package**

- enforces logical structuring of your code, breaking it into smaller, manageable modules (reusable).
- lets you distribute your code easily, either within your team or to the public.

What is a Python package?

A **python package**

- enforces logical structuring of your code, breaking it into smaller, manageable modules (reusable).
- lets you distribute your code easily, either within your team or to the public.
- includes metadata for versioning, allowing users to install specific versions of your package, and it helps manage dependencies automatically.

What is a Python package?

A **python package**

- enforces logical structuring of your code, breaking it into smaller, manageable modules (reusable).
- lets you distribute your code easily, either within your team or to the public.
- includes metadata for versioning, allowing users to install specific versions of your package, and it helps manage dependencies automatically.
- makes it easier for others to understand, use, and contribute to your work.

File Tree


```
my_project
├── my_project/
│   ├── __init__.py
│   └── ... (your project code)
├── data/ (optional)
├── output/ (optional)
├── requirements.txt or requirements.yaml (optional)
├── setup.py or pyproject.toml
├── .gitignore
│
│  (You will populate the files below next week)
│
├── tests/
├── examples/ or tutorials/
├── docs/
├── README.md
└── LICENSE
```

Package directory

Here goes your whole python code. It is called like your **package name**.

The `__init__.py` is an **empty** file that is needed by python to recognize the directory as a package.

```
my_project
└── my_project/
    ├── __init__.py
    └── ... (your project code)
```

`local` directories

These are directories like your **input** or **output** data directories

These are files that can be **easily recreated** or **downloaded** from some other source (might be automated).

You might also create a **`scratch/`** directory which includes scripts that you use for experimentation

These directories should be included in the `.gitignore` file.

```
my_project
├── data/
├── output/
└── .gitignore
```

`tests` directory

Reserved to **automated test code**, testing the **correctness** of your program.

```
my_project
└── tests/
```

`setup.py` and `pyproject.toml` file

Automated **setup instructions** on how to **build and install** your package.

`pyproject.toml` is the **newer version** of `setup.py` that is **recommended**.

```
my_project
└── setup.py or pyproject.toml
```

`requirements.txt` file

This is an **older** version of how to **manage dependencies** (other packages you might use in your project).

It included a **list of all the packages** your package needs to work and which **version** it uses.

It is now directly included `pyproject.toml` and therefore, it is **not needed anymore**.

```
my_project
└── requirements.txt or
requirements.yaml
```

`examples` directory

Reserved for some examples/ tutorials, which are explanations on how to use your project. How to get it running and how to start working on it. It may include some **jupyter-notebooks**.

```
my_project
└── examples/ or tutorials/
```

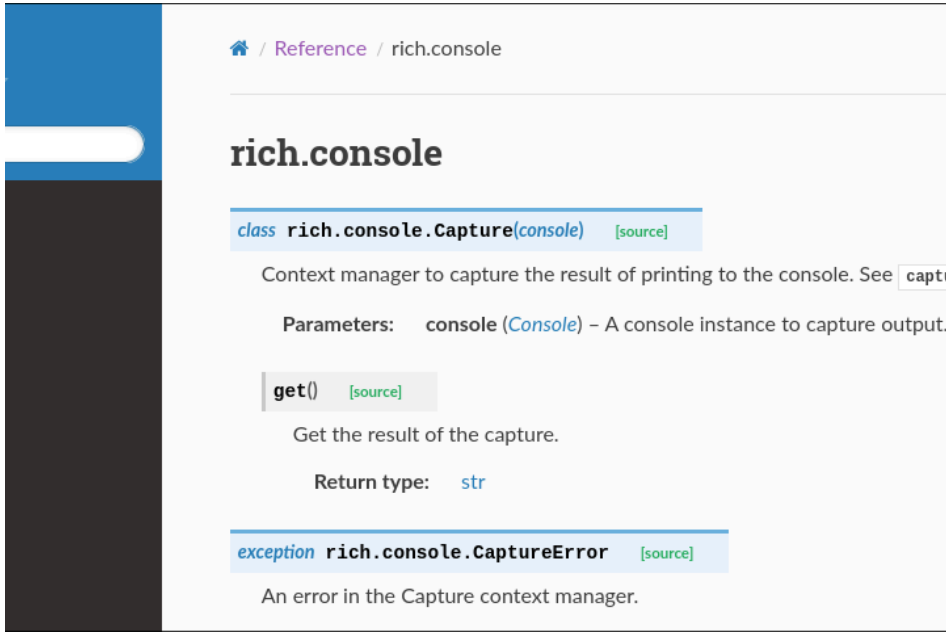
docs directory

Documentation about the usage, behaviour and side effects of all your functions, classes, ...

This is often **auto generated** from your **docstrings** (using e.g. Sphinx).

Historically, they were often **man** pages (just text files), today they are more commonly websites.

```
my_project
└── docs/
```



Docs vs Tutorial

Docs:

- Very technically
- Detailed
- Not following a goal or storyline

Tutorial:

- Very approachable for first time users
- Not too technical
- Often show the user an example

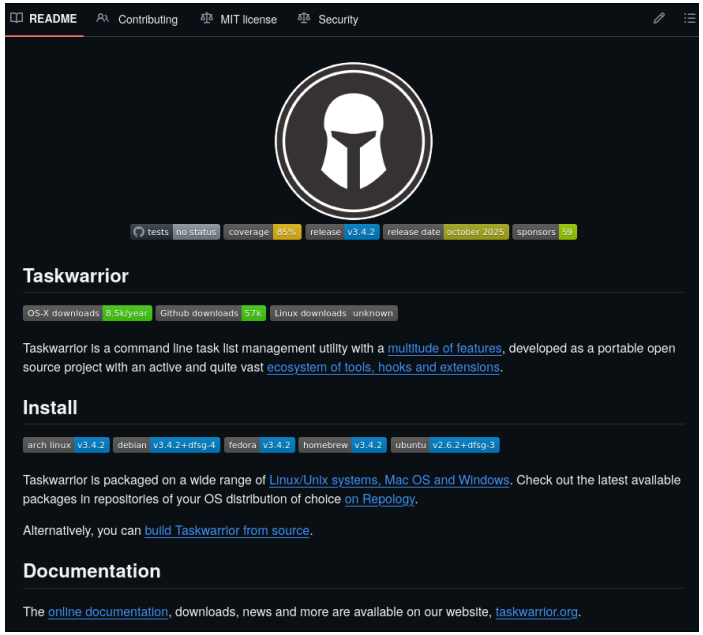
`README.md` file

The **frontpage** of your project. It is the first thing users will refer to.

It is written in **Markdown** and includes:

- Short Description
- Installation Instructions
- Usage Examples
- Explanation of the Project structure
- Dependencies
- License
- Contribution Guidlines
- Contact Information

```
my_project
└── README.md
```



`LICENSE` file

LEGAL DISCLAIMER: We are not legal advisors. When building something important please consider professional advice.

For small projects it doesn't matter that much

Example Licenses:

- **MIT**: Project can be used however people want (even commercially)
- **Apache**: Similar to MIT, but authors must be stated
- **GPLv3**: Adheres to the **FSF Four Freedoms**
- AGPL: Similar to GPLv3 but fixes the SAAS abuse

For more information see: **https://choosealicense.com**

MiniTask: Create a basic project structure

1. Someone in the team do this for the main project. All others create a new git repository (`git init`)
2. Recreate this directory structure
3. Create in `my_module.py` a small function

```
my_project
├── my_project/
│   ├── __init__.py
│   └── my_module.py
├── data/
├── setup.py or pyproject.toml
└── .gitignore
```

Small Function example:

```python
def magic_number():
  return 5
```

Package Management

Installing Packages

To install packages you need a **package manager**. A package manager **automatically downloads** python packages for you, manages their **dependencies**, **versions** for you and makes them available for you to use.

Installing Packages

To install packages you need a **package manager**. A package manager **automatically downloads** python packages for you, manages their **dependencies**, **versions** for you and makes them available for you to use.

Over the years, more and more package managers come up.

- **pip**
- Anaconda
- uv

Package Managers

Pip

- Built into Python (original version)
- Is Python only
- More manual

Package Managers

Pip

- Built into Python (original version)
- Is Python only
- More manual

Conda (Anaconda)

- Requires Anaconda or Miniconda installed
- Supports multiple languages like Python, R, C, Java
- More automatic

Package Managers

Pip

- Built into Python (original version)
- Is Python only
- More manual

Conda (Anaconda)

- Requires Anaconda or Miniconda installed
- Supports multiple languages like Python, R, C, Java
- More automatic

UV

- Very new and very fast
- Fixes some issues of the other twos

Pip

To install a package you can run

```
pip install <package name>
```

This will search for the package on the **PyPi** Package Registry

Pip

To install a package you can run

```
pip install <package name>
```

This will search for the package on the **PyPi** Package Registry

For example

```
pip install numpy # install the latest version
pip install numpy==2.4.2 # install specific version (2 versions behind latest)
```

Need for Virtual Environments

This would install packages **globally**

Why is this **very bad**?

Need for Virtual Environments

This would install packages **globally**

Why is this **very bad**?

- Sometimes you need different versions for different projects
- Packages influencing each other
- Packages should be closely bound to your project

Need for Virtual Environments

This would install packages **globally**

Why is this **very bad**?

- Sometimes you need different versions for different projects
- Packages influencing each other
- Packages should be closely bound to your project

Solution:

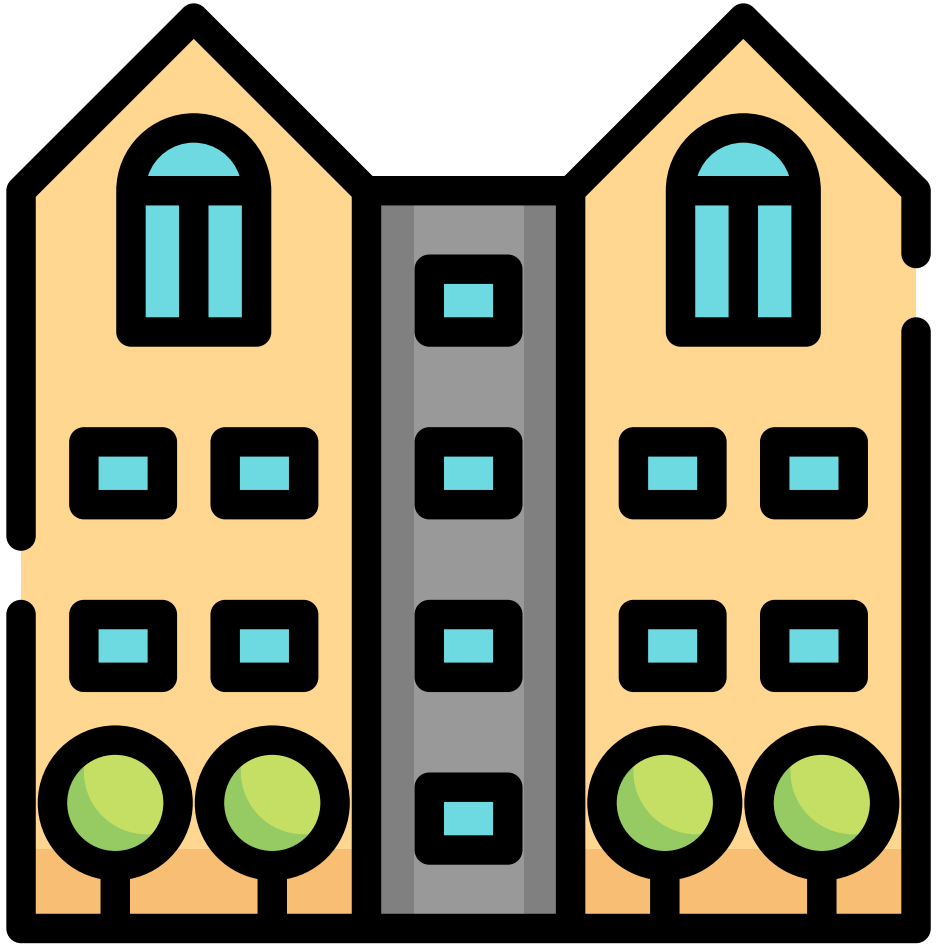
Virtual Environments encapsulate packages to their respective project and python version.

This avoids dependency conflicts and make projects more reproducible.

Analogy

Your computer is like an appartment building.

Every single appartment is a **virtual environment**



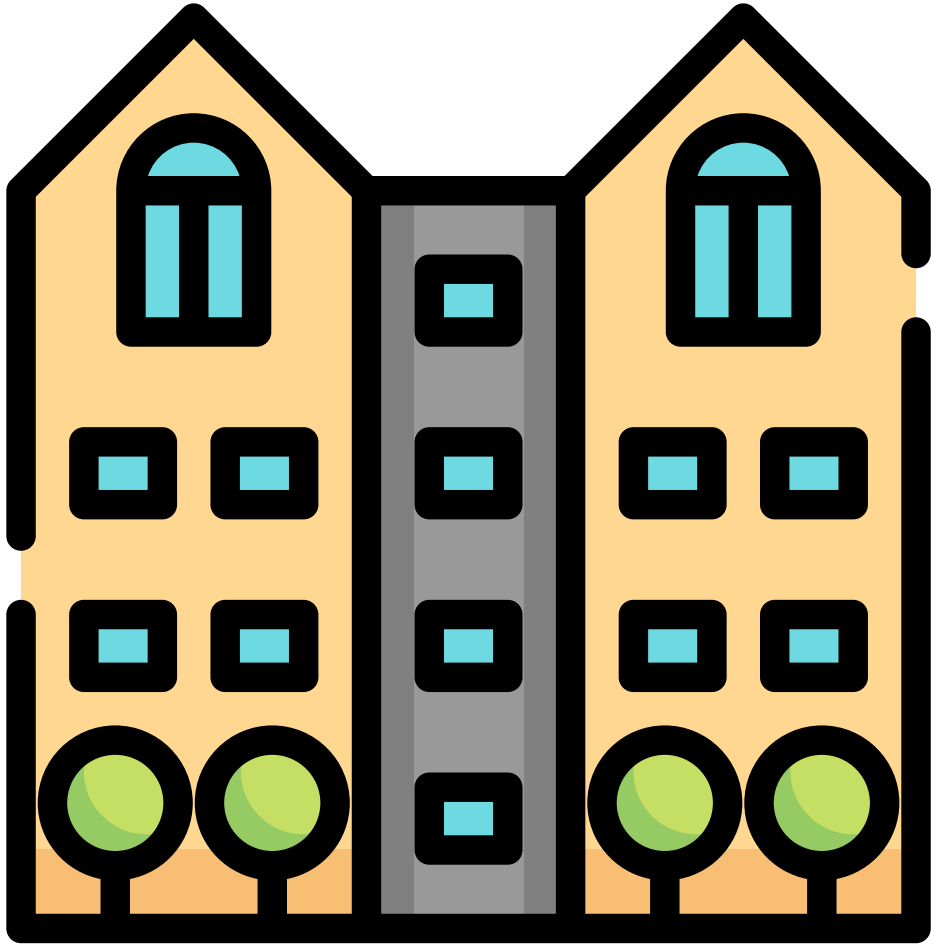
Analogy

Your computer is like an appartment building.

Every single appartment is a **virtual environment**

Breaking a single appartment doesn't affect the others

Although **breaking** the **global building** will affect all appartments.



Creating a virtual environment

To create a virtual environment you can run in your project folder

```
python -m venv env_name
```

Using a virtual environment

To then actually go into your virtual environment you can run

Mac/Linux:

```
source env_name/bin/activate
```

Windows CMD:

```
env_name\Scripts\activate.bat
```

Windows PowerShell:

```
env_name\Scripts\Activate.ps1
```

To go out of it again, run

```
deactivate
```

Packages in venv

You can then install with **pip** packages into the virtual environment by running

```
pip install numpy # install the latest version
pip install numpy==2.4.2 # install specific version (2 versions behind latest)
```

These packages stop existing for other projects when not using the same virtual environment.

MiniTask: Create your own virtual enviornment

1. Create a virtual environment
2. Switch into the virtual environment
3. Try installing some packages (e.g. `pip install numpy==2.4.2`) that you might need for your project
4. Try to import this package by running in python

```
import numpy
```

Conda Virtual Environments

Conda has it's own virtual environment management system

Creating a new environment:

```
conda create -n env_name python=3.12
```

📝 Note:

With anaconda you can also manage multiple python versions easily, which is not possible with venv

Conda Virtual Environments

Activate it:

```
conda activate env_name
```

Deactivating it:

```
conda deactivate
```

Remove it:

```
conda remove --name env_name
```

Installing packages with conda

You can install packages with conda by running

```
conda install numpy
```

What we have achieved so far

- We have the **minimum file structure** to have a package
- We can **use packages** inside of our package

What we have achieved so far

- We have the **minimum file structure** to have a package
- We can **use packages** inside of our package

What is **missing**: Making our package **installable**

What we have achieved so far

- We have the **minimum file structure** to have a package
- We can **use packages** inside of our package

What is **missing**: Making our package **installable**

We need to create a proper **`pyproject.toml`**

Content of the pyproject.toml

Content of the pyproject.toml


```toml
# Describes what build system to use while building the package
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

Content of the pyproject.toml


```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

# Describes general information about your package
[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
```

Content of the pyproject.toml


```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
# Optionally specifies where the readme is
readme = "README.md"
```

Content of the pyproject.toml


```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
readme = "README.md"
# Defines the dependencies of the package
requires-python = ">=3.12"
dependencies = ["numpy==2.4.2"]
```

Content of the pyproject.toml

```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["numpy==2.4.2"]

# Specifies where the system can find your code and what should be included in
your package
[tool.setuptools.packages.find]
where = [""]
include = ["my_project"]
```

Content of the pyproject.toml


```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["numpy==2.4.2"]

[tool.setuptools.packages.find]
where = [""]
include = ["my_project"]
```

Installing your package

To install the package run

```
pip install .
```

To install it in **development mode** run

```
pip install -e .
```

This will **automatically update** the installed package when you make **changes to your project** after you import the package again.

MiniTask: Install your own package

1. Create a **`pyproject.toml`** file
2. **Install** the project while being in your **virtual environment**
3. **Test** your installation by running the following command
4. **Try** to import your package from somewhere else on your computer (e.g. python shell, jupyter notebook, another script, ...) and try to import it while not being in the project folder / virtual environment

```python
from my_project.my_module import magic_function

magic_function()
```

Paths

Types of paths

There are two types of paths

Types of paths

There are two types of paths

Absolute Paths:

Windows:

```
C:\Users\Alexis\Documents\my_project
```

MacOS/Linux:

```
/home/Alexis/Documents/my_project
```

Types of paths

There are two types of paths

Absolute Paths:

Windows:

```
C:\Users\Alexis\Documents\my_project
```

MacOS/Linux:

```
/home/Alexis/Documents/my_project
```

Relative Paths:

Windows:

```
Documents\my_project
```

MacOS/Linux:

```
Documents/my_project
```

Problems

Operating System incompatibilites and **hardcoded absolute paths** cause issues

Problems

Operating System incompatibilites and **hardcoded absolute paths** cause issues

For example

```
C:\Users\Alexis\Documents\my_project\
```

```
/home/Alexis/Documents/my_project
```

This is inherently **not compatible**. Also not **every user** on a computer is called **Alexis**.

Solution

There is a built-in python package called **pathlib**. It will clear out differences between operating systems

```
from pathlib import Path

p = Path("data")

filepath = p / "my_data.csv"
other_filepath = p / "audio_data" / "file.mp3"
```

Project Paths

To have certain paths easily accessible (like your **data** directory) you can use the `__init__.py` file.

In there you can add something like that

Project Paths

```python
from pathlib import Path

# Get the path of the package directory
PACKAGE_DIR = Path(__file__).resolve().parent

# Get the path of the project directory (one level up)
PROJECT_DIR = PACKAGE_DIR.parent

# Define specific directories
DATA_DIR = PROJECT_DIR / "data"
OUTPUT_DIR = PROJECT_DIR / "output"

# Optionally, ensure the folders exist
DATA_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

Why this works

`Path(__file__)` always resolves to the **location of the file it's written in**, not where you run Python from.

Why this works

`Path(__file__)` always resolves to the **location of the file it's written in**, not where you run Python from.

The command:

```
from my_project import DATA_DIR

print(DATA_DIR)
```

Why this works

`Path(__file__)` always resolves to the **location of the file it's written in**, not where you run Python from.

The command:

```
from my_project import DATA_DIR

print(DATA_DIR)
```

should point to the project's data folder, no matter:

- which computer you're on
- which directory you're in
- whether you run a script, a notebook, or the interactive shell

MiniTask: Prove your paths work everywhere

1. Add the path definitions from the previous slide to `__init__.py`
2. Open a Python shell **from a completely different folder**
3. Run the code below; does it print the correct path to your data folder?
4. Now try creating a file inside `DATA_DIR` from that remote location. Can you read it back from your project?
5. **Bonus:** What happens if you delete the `data/` folder and import `DATA_DIR` again?

Team Reflection

Energizer

You pass the ball around and when you have the ball please answer the following two questions

1. Where are you still struggling with regarding the theory
2. What do you think have you understood quite well

Status Reflection

You have a paper with a line in front of you and some dots (stickers, markers, magnets, ...)

Make notes in your Project Timeplan

1. Put a dot on a line where you **currently** are in the project timeline. **Left**: You have not started, **Right**: You can submit today
2. Discuss in your group why there might be differences in your perception of the progress
3. Update your current Timeplan? What have you achieved so far?
4. What are the next milestones? How can you achieve these milestones?

Challenges

Everyone should take a **pen** and a **post it**.

1. You have **3min** to write down what you think are some **risks, fears or road blocks** of your project

Challenges

Everyone should take a **pen** and a **post it**.

1. You have **3min** to write down what you think are some **risks, fears or road blocks** of your project
2. You have another **3min** to **pair** up with **another person** in your team and share your risks, fears and road blocks. Can you find **new challenges**?

Challenges

Everyone should take a **pen** and a **post it**.

1. You have **3min** to write down what you think are some **risks, fears or road blocks** of your project
2. You have another **3min** to **pair** up with **another person** in your team and share your risks, fears and road blocks. Can you find **new challenges**?
3. You have **3min** to cluster all your risks, fears and road blocks together.

Challenges

Everyone should take a **pen** and a **post it**.

1. You have **3min** to write down what you think are some **risks, fears or road blocks** of your project
2. You have another **3min** to **pair** up with **another person** in your team and share your risks, fears and road blocks. Can you find **new challenges**?
3. You have **3min** to cluster all your risks, fears and road blocks together.
4. You have another **3min** time to think about: What are **potential solutions**? What are **checkpoints** to realize that you are **on the right track**?

Check Out

Reflect and Discuss in your group

1. What where the **key insights** of this reflection session?
2. What is the **immediate next step**?
3. Every team shares with the other teams what their next step is